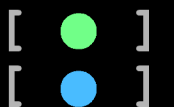


\$./ TC1031.100. Modelación de sistemas mínimos y arquitecturas computacionales

<LMC/> Ensamblador de código en Python

Dante Martínez Domínguez + Ruben Caro Romero + Santiago Cavazos Nuñez



// iniciar.listos

Idea general

Objetivo del código



El programa recibe un archivo de tipo .txt escrito con mnemónicos y etiquetas:

- Lo analiza en dos pasadas.
- Genera un archivo con las 100 posiciones de memoria.
- Lo conecta con el intérprete de LMC.

```
# Programa 1: suma dos números.  
# Entradas: A, B  
# Salida: A + B  
# Ejemplo: entradas 5 y 6 -> salida 11
```

	INP	STA A	INP	ADD A	OUT	HLT
A	DAT 000					

			≡ solucion1.txt
1	00	901	
2	01	306	
3	02	901	
4	03	106	
5	04	902	
6	05	000	
7	06	000	
8	07	000	
9	08	000	
10	09	000	
11	10	000	
12	11	000	
13	12	000	
14	13	000	
15	14	000	
16	15	000	

// El fundamento

ejecutar_lmc.py

1

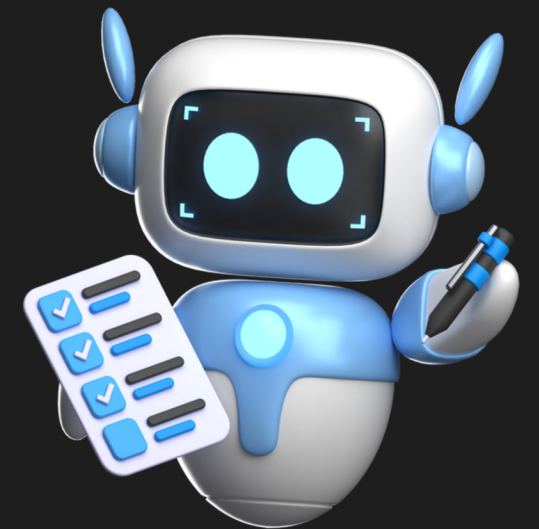
LEER ARCHIVO

- Se definen los mailboxes (0 al 99).
- Determinar partes de cada línea.
- Determinar dirección e instrucción correspondiente.

2

LITTLE MAN COMPUTER

- Se definen los mnemónicos e instrucciones.
- Se agrega soporte para subrutinas (4xx y 999).
- Se validan *opcodes* desconocidos (raise ValueError).



```
// my_solution - three_pillars()
```

Primer pasada

```
def primera_pasada(archivo_entrada):
```

Lee el archivo línea por línea.

Elimina comentarios y espacios

Identificando:

- **Mnemónicos**
- **Operandos**
- **Etiquetas**

Asigna una dirección de memoria a cada instrucción.

```
def primera_pasada(archivo_entrada):
```

```
    simbolos = {}  
    instrucciones = []  
    direccion = 0  
    ...
```

```
    # Caso 1: la línea empieza directamente con un mnemónico.
```

```
    if partes[0] in mnemonicos:  
        mnemonico = partes[0]  
        operando = partes[1] if len(partes) > 1 else None
```

```
    # Caso 2: la línea empieza con una etiqueta.
```

```
    else:  
        etiqueta = partes[0]  
        mnemonico = partes[1]  
        operando = partes[2] if len(partes) > 2 else None
```

```
        simbolos[etiqueta] = direccion # Guarda la dirección donde aparece la etiqueta.
```

```
    ...
```

```
    return simbolos, instrucciones
```

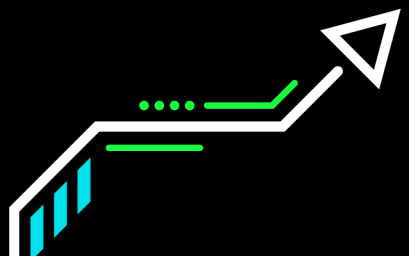
```
// my_solution - second_pillar()
```

Segunda pasada

```
def segunda_pasada(simbolos, instrucciones):
```

- Utilizar la tabla de símbolos.
- Convertir cada mnemónico a código máquina.
- Si existe un operando, obtener la dirección.

```
def segunda_pasada(simbolos, instrucciones):  
    memoria = [0] * 100  
  
    for direccion, mnemonico, operando, numero_linea in instrucciones:  
        if mnemonico in sin_operando:  
            ...  
            memoria[direccion] = int(mnemonicos[mnemonico])  
  
        elif mnemonico in con_operando:  
            ...  
            direccion_operando = simbolos[operando]  
            memoria[direccion] = int(mnemonicos[mnemonico]) * 100 + direccion_operando  
  
        elif mnemonico == "DAT":  
            if operando is None:  
                memoria[direccion] = 0  
            else:  
                try:  
                    memoria[direccion] = int(operando)  
  
    return memoria
```



```
// pricing.tiers
```

Validación de errores

SE VERIFICA SI:

`raise ValueError()`

- Etiquetas duplicadas.
- Etiquetas inexistentes.
- Mnemónicos inválidos.
- Operandos faltantes.
- Memoria fuera de rango.



```
if len(partes) < 2:
    raise
ValueError(f"Falta mnemónico después de
la etiqueta en línea {numero_linea}:
{etiqueta}")

mnemonico = partes[1]
operando = partes[2] if
len(partes) > 2 else None

if etiqueta in
simbolos: # Detecta etiquetas
repetidas.

    raise
ValueError(f"Etiqueta duplicada en
línea {numero_linea}: {etiqueta}")
```



```
if mnemonico not in
mnemonicos: # Valida que el mnemónico
sí exista.

    raise
ValueError(f"Mnemónico desconocido en
línea {numero_linea}: {mnemonico}")

instrucciones.append((direccion,
mnemonico, operando, numero_linea)) #
Guarda la instrucción procesada para la
segunda pasada.

    direccion += 1 # Avanza a
la siguiente casilla de memoria.

if direccion > 100:
    raise ValueError("El
programa excede las 100 casillas
disponibles")
```

```
// my_solution - three_pillars()
```

Código original (Primer alternativa)

●●● ~ / Suma de dos números

```
# Programa 1: suma dos números.  
# Entradas: A, B  
# Salida: A + B  
# Ejemplo: entradas 5 y 6 ->  
salida 11
```

```
INP  
STA A  
INP  
ADD A  
OUT  
HLT
```

```
A      DAT 000
```

// Código en Python

```
# Programa 1  
ensamblar("programa1.txt",  
"solucion1.txt")  
pe = lm.leertxt("solucion1.txt")  
print("El resultado del programa 1 es:",  
lm.ejecutar_lmc(pe, [5, 6]))
```

// Resultado en consola

El resultado del programa 1 es: [11]

Código original (Segunda alternativa)

● ● ● ~ / Producto de dos números

INP
STA A
INP
STA B

LDA ZERO
STA RES
LDA B
STA CONT

LOOP LDA CONT
BRZ FIN
LDA RES
ADD A
STA RES
LDA CONT
SUB ONE
STA CONT
BRA LOOP

FIN	LDA RES
	OUT
	HLT
A	DAT 000
B	DAT 000
RES	DAT 000
CONT	DAT 000
ONE	DAT 001
ZERO	DAT 000

// Resultado en consola

El resultado del programa 2 es: [30]

Código original (Tercer alternativa)

~ / Sumatoria de 1 hasta n

```
                INP
                STA N

                LDA ZERO
                STA SUM
                LDA ONE
                STA I

LOOP            LDA N
                SUB I
                BRP BODY
                BRA END

BODY            LDA SUM
                ADD I
                STA SUM

                LDA I
                ADD ONE
                STA I
                BRA LOOP
```

```
END            LDA SUM
                OUT
                HLT
```

```
N              DAT 000
SUM            DAT 000
I              DAT 000
ONE            DAT 001
ZERO           DAT 000
```

// Resultado en consola

El resultado del programa 3 es: [15]

```
// El final...
```

Conclusión

```
●●● ~/stats - run --since=2026
```

```
$ ./stats --aggregate // binario es difícil 0 1 0 1
```

01 EL BINARIO ES LA BASE

10 VALORAR CADA LÍNEA DE CÓDIGO
Funciones y subrutinas...

11 EL COMPILADOR HUMANO
El pseudocódigo.

100 DE LA ENIAC A LA IA
Necesidad de adaptación cte.